# PBA: 6. Summarizing Bivariate Relationships & Tidying Data

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

# Where are we? Where are we going?

- **Where we have been**
  - Week 1: R and RStudio, functions and packages, Quarto
  - Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`
  - Week 3: data wrangling with `dplyr`. Rows, columns, and groups, then barplots from the summaries
  - Week 4: causality. Causal effects, experiments, and observational studies
  - Week 6: causality II. DAGs and which variables to control for

# Where are we? Where are we going?

- **Where we have been**
  - Week 1: R and RStudio, functions and packages, Quarto
  - Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`
  - Week 3: data wrangling with `dplyr`. Rows, columns, and groups, then barplots from the summaries
  - Week 4: causality. Causal effects, experiments, and observational studies
  - Week 6: causality II. DAGs and which variables to control for
- **Today: relationships, importing, and tidying data**
  - Z-scores and correlation: is 60% a lot, and do two variables move together?
  - Our own function, `z_score()`, applied to many columns with `across()`
  - Pivoting longer and joining: reshaping and combining data sets

# Where are we? Where are we going?

- **Where we have been**
  - Week 1: R and RStudio, functions and packages, Quarto
  - Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`
  - Week 3: data wrangling with `dplyr`. Rows, columns, and groups, then barplots from the summaries
  - Week 4: causality. Causal effects, experiments, and observational studies
  - Week 6: causality II. DAGs and which variables to control for
- **Today: relationships, importing, and tidying data**
  - Z-scores and correlation: is 60% a lot, and do two variables move together?
  - Our own function, `z_score()`, applied to many columns with `across()`
  - Pivoting longer and joining: reshaping and combining data sets
- **Where we are going**
  - Next class (Oct 29): prediction and iteration. Then regression and uncertainty

# Roadmap

0. Causality review

1. Z-scores and standardization

2. Correlation

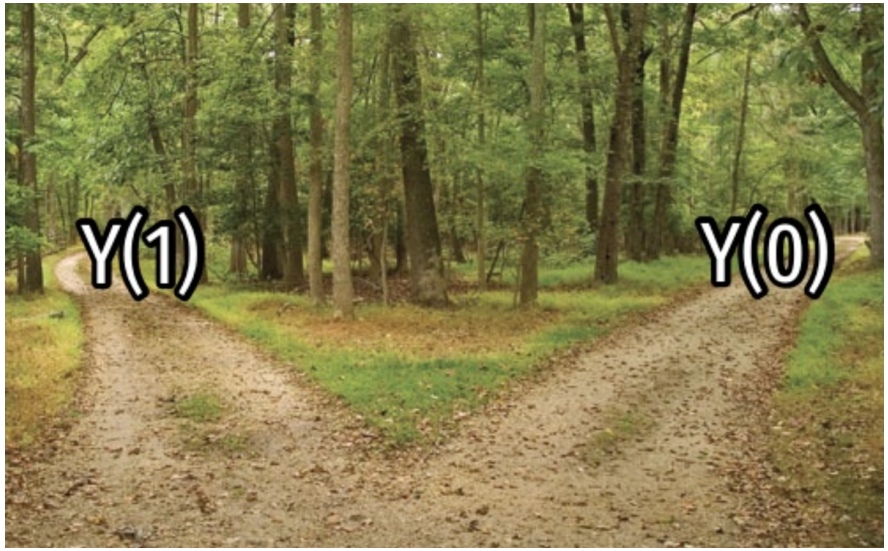
3. Writing our own functions

4. Pivoting data longer

5. Joining data sets

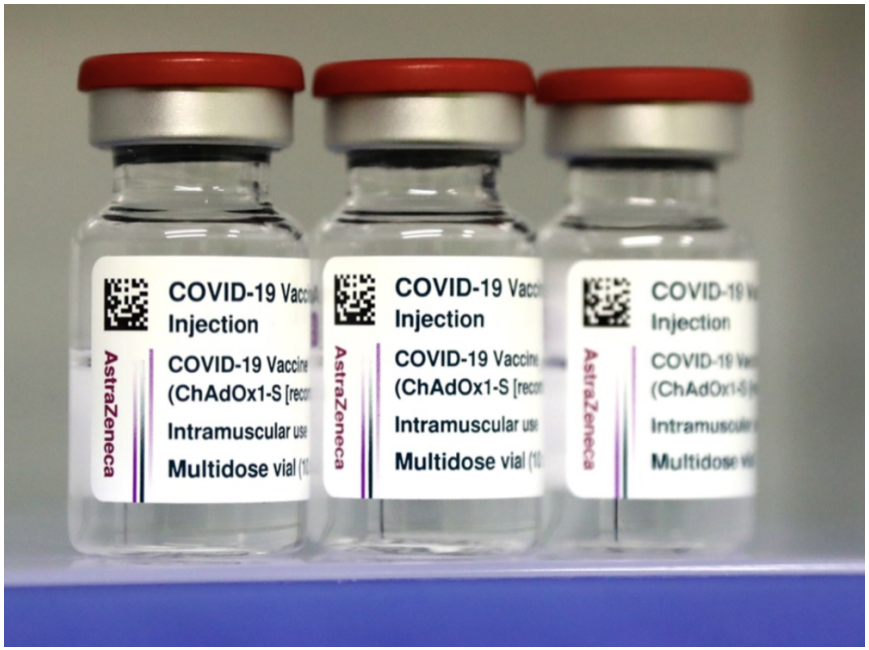
# o/ Causality review

# Potential Outcomes *Redux*




- Potential outcomes:
  - $Y_i(1)$ is the value that the outcome would be if we gave unit $i$ treatment and changed nothing else about them.
  - $Y_i(0)$ is the value that the outcome would be if we gave unit $i$ **no treatment** and changed nothing else about them.
  - Not the **possible values** of the outcome.

# COVID-19 Vaccine Trials




- **Treatment**: $T_i = 1$ if vaccinated, $T_i = 0$ if not.
- **Outcome**: $Y_i = 1$ if acquired COVID after 12 weeks, $Y_i = 0$ if not.

Q1. What are the potential outcomes $Y_i(1)$ and $Y_i(0)$?

Q2. Why not compare early volunteers for the vaccine with the overall population?

# 1/ Z-scores and standardization

A county in the US reported that, as of September 2022, **60%** of its residents had already received at least one COVID-19 vaccine dose.

That sounds impressive. But:

**Q1.** Compared with other counties, is 60% high?

**Q2.** Would the answer change if the 60% were among residents aged 65+?

# COVID Vaccination Data

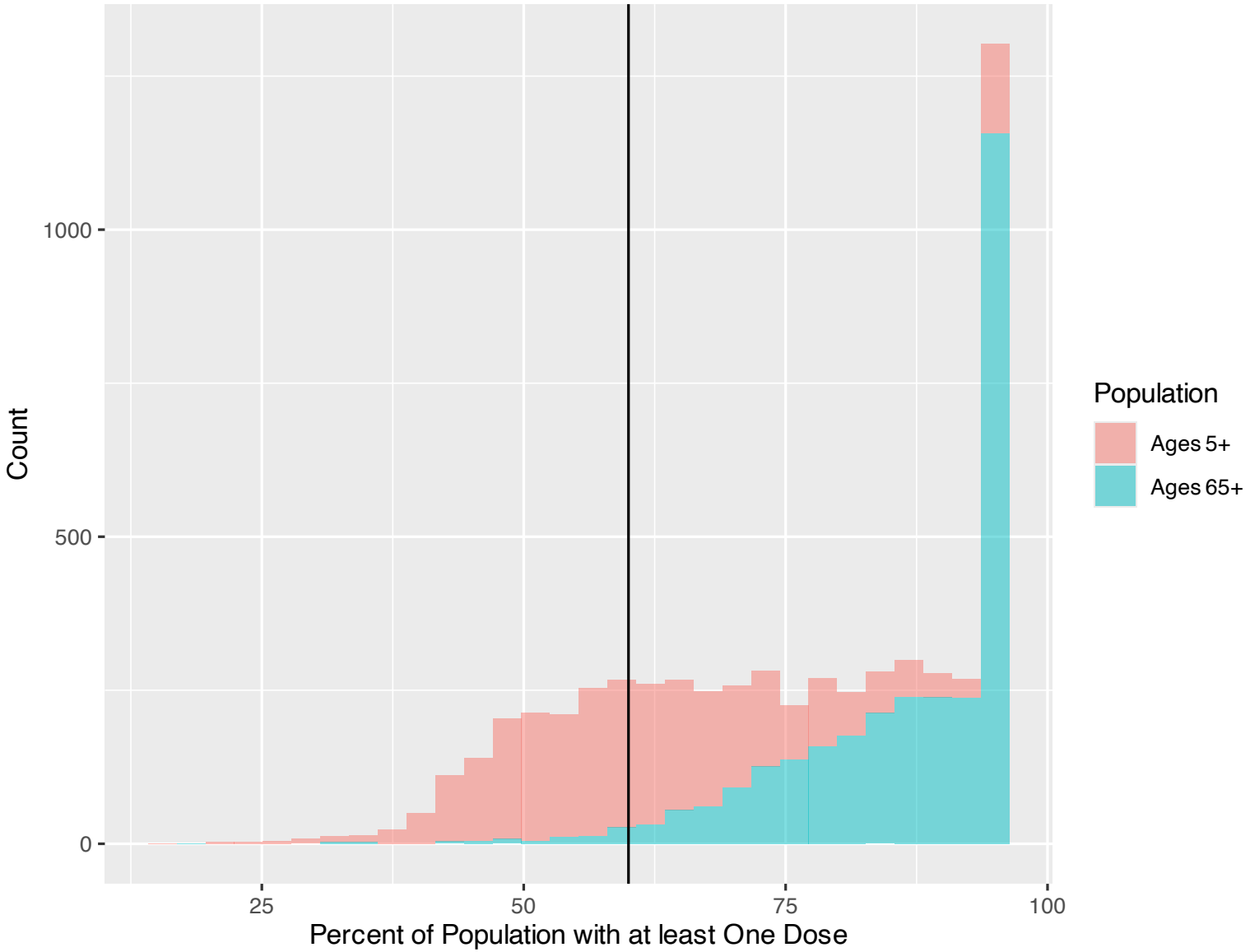
- Data on county-level vaccination rates (CDC, as of Sep 28, 2022) and presidential voting (MIT Election Lab):

```
# Don't forget to load tidyverse and ggplot2!
# today's three data sets are also in the course package pbadata (see the top of the R script)
> covid_data <- read_csv(url("https://bit.ly/3FOLohq")); covid_data

# A tibble: 3,114 × 8
   fips  county           state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
   <chr> <chr>            <chr>              <dbl>               <dbl>             <dbl>
 1 26039 Crawford County  MI                  55.7                77.3              31.2
 2 40015 Caddo County     OK                  83.3                95                30.3
 3 17007 Boone County     IL                  71.1                94.5              35.1
 4 12055 Highlands County FL                  68.9                93.7              24.7
 5 34029 Ocean County     NJ                  71                  95                32.1
 6 01067 Henry County     AL                  58.5                85.5              18.2
 7 27037 Dakota County    MN                  81                  95                49.5
 8 27115 Pine County      MN                  56.5                85                31.7
 9 51750 Radford city     VA                  41.5                73.8               1.79
10 22009 Avoyelles Parish LA                  59.7                80.1              21.9
# ℹ 3,104 more rows
# ℹ 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
# ℹ Use `print(n = ...)` to see more rows
```

  - `read_csv()` usually reads numbers with leading zeros (01067) as text; `read.csv()` always reads them as numbers (1067).
  - For ID columns, set the type yourself with the `col_types` argument: `read_csv(..., col_types = cols(fips = col_character()))`

# Is 60% Vaccinated A Lot?



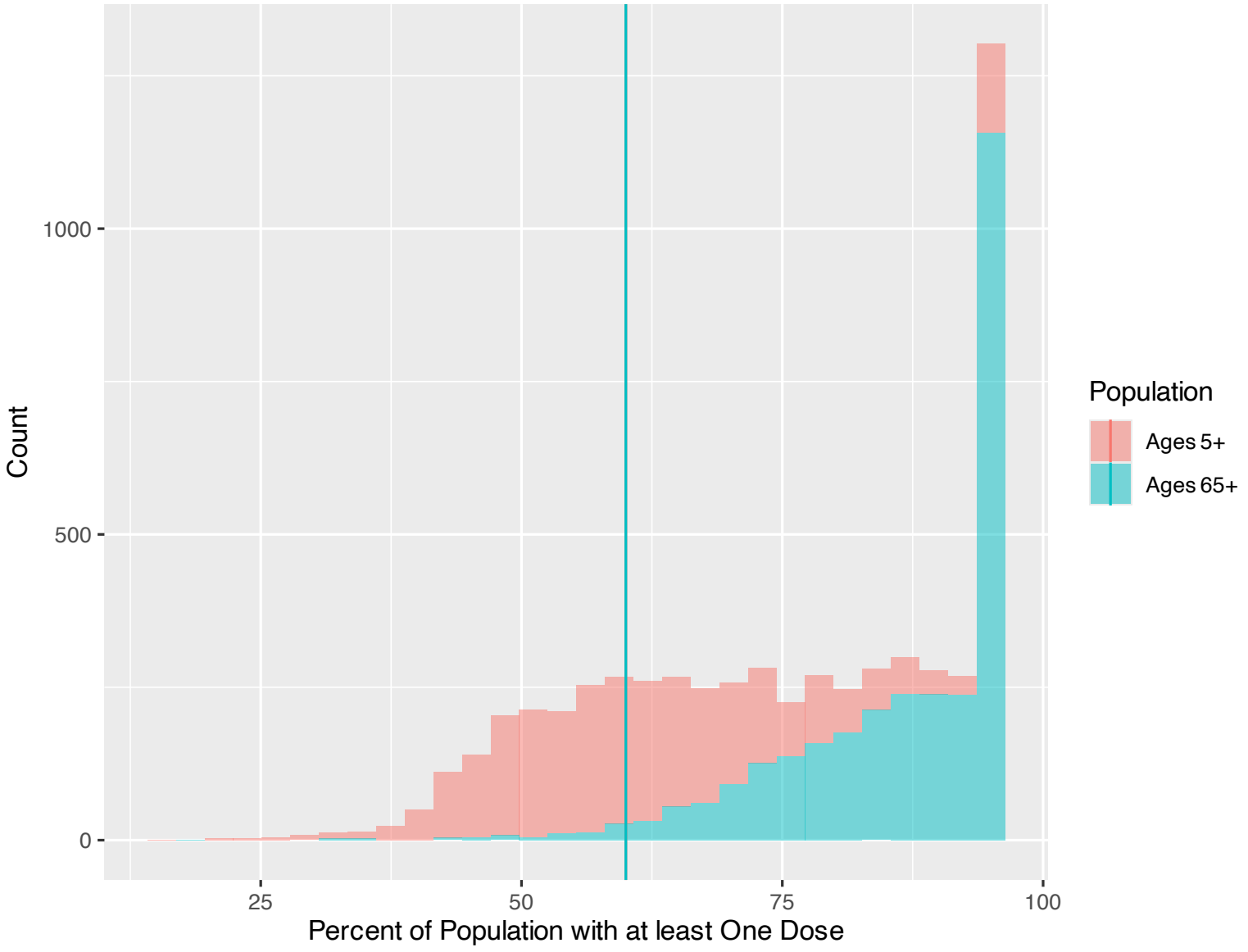
# How Large is Large?

- How large 60% vaccinated is depends on the distribution!
  - Clear to see from the histogram.
  - Middling for the 5+ group, but very low for the 65+ group.

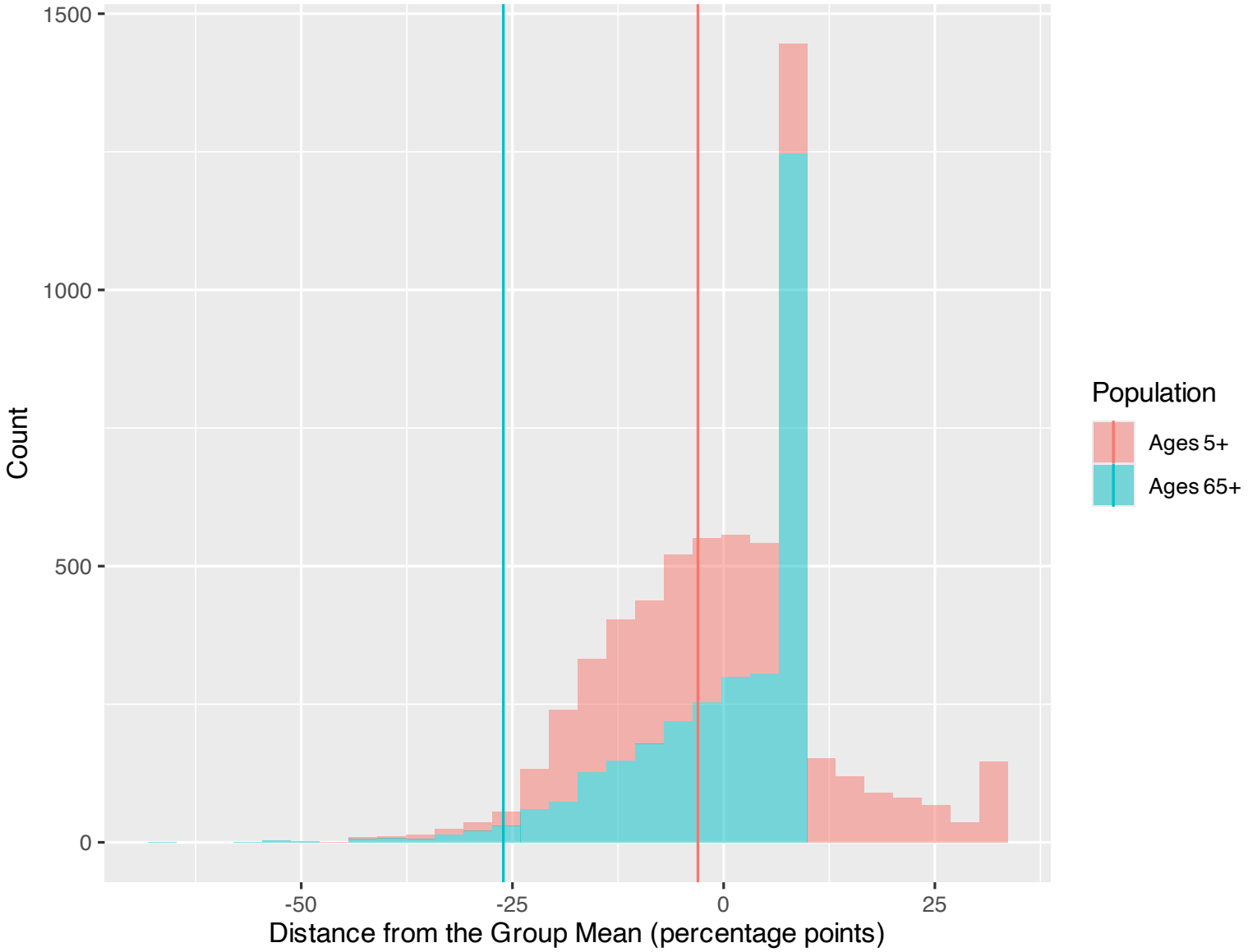
- Can we transform the values of our variables onto a **common scale**?
  - "Large" means the same thing for any variable: far from the typical value, relative to how much values usually vary.

- Yes, with two transformations:
  - **Centering**: subtract the mean of the variable from each value.
  - **Scaling**: divide the deviations from the mean by the standard deviation.

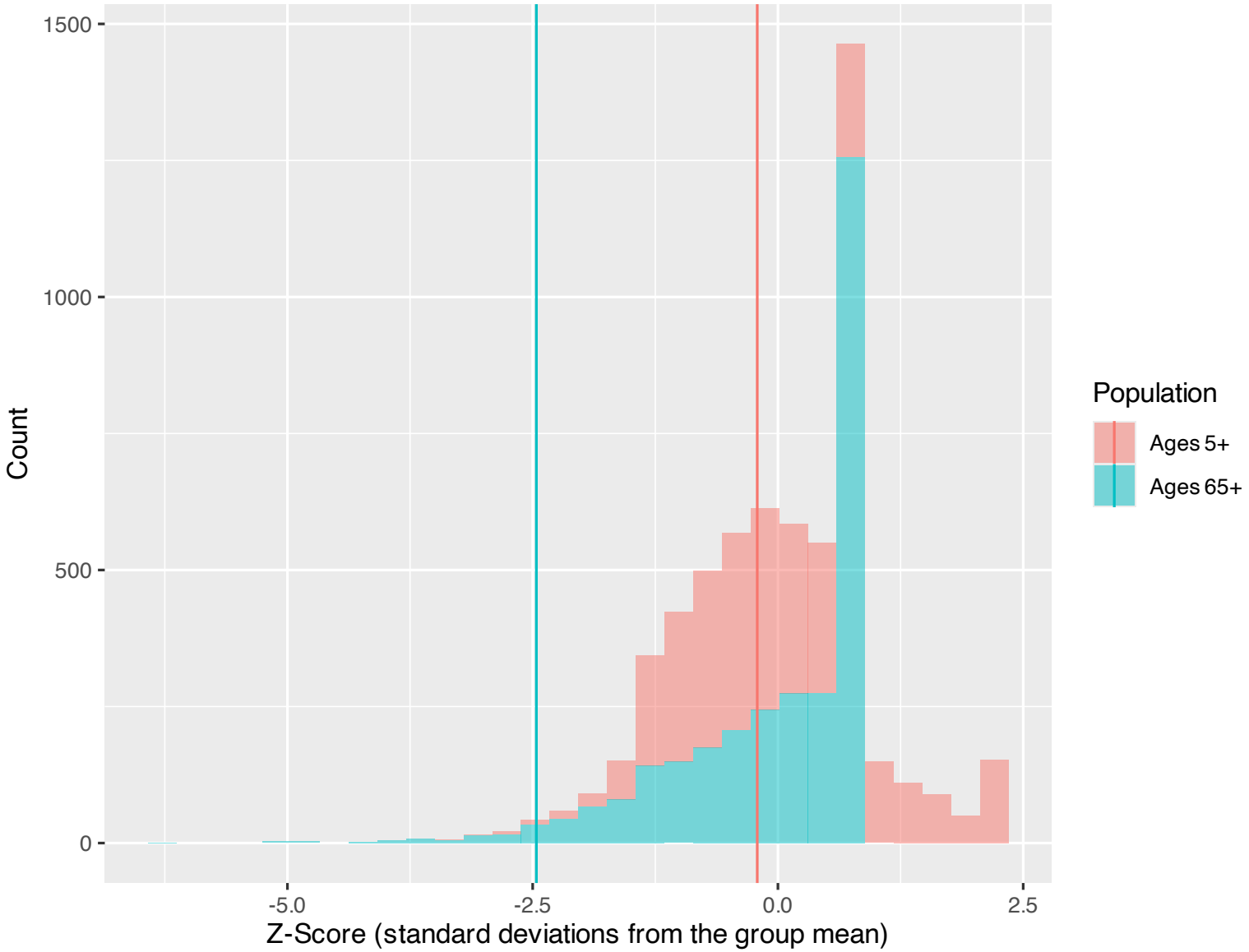
# Original Distributions



# Centered Distributions



# Centered and Scaled Distributions



# Z-Scores

- Centering tells us immediately if a value is above or below the mean.
- Scaling tells us how many standard deviations away from the mean it is.
- Combine them with the **z-score** transformation:

$$\text{z-score of } x_i = \frac{x_i - \text{mean of } x}{\text{standard deviation of } x}$$

- Useful heuristic: data more than 3 SDs away from the mean are 'rare.'
  - An outlier rule different from the box plot's (Week 2), which uses the whiskers to flag points more than $1.5 \times$ IQR beyond the box.

# Z-Score Example: Centering

```
> covid_data |>
    mutate(one_dose_centered = one_dose_5plus_pct -
             mean(one_dose_5plus_pct, na.rm = T)) |>
    select(fips:state, one_dose_5plus_pct, one_dose_centered)

# A tibble: 3,114 × 5
   fips  county           state one_dose_5plus_pct one_dose_centered
   <chr> <chr>            <chr>              <dbl>             <dbl>
 1 26039 Crawford County  MI                  55.7             -7.35
 2 40015 Caddo County     OK                  83.3             20.2
 3 17007 Boone County     IL                  71.1              8.05
 4 12055 Highlands County FL                  68.9              5.85
 5 34029 Ocean County     NJ                  71                7.95
 6 01067 Henry County     AL                  58.5             -4.55
 7 27037 Dakota County    MN                  81               17.9
 8 27115 Pine County      MN                  56.5             -6.55
 9 51750 Radford city     VA                  41.5            -21.6
10 22009 Avoyelles Parish LA                  59.7             -3.35
# ℹ 3,104 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# Z-Score Example: Centering & Scaling

```
> covid_data |>
    mutate(one_dose_z = (one_dose_5plus_pct - mean(one_dose_5plus_pct, na.rm = T)) /
             sd(one_dose_5plus_pct, na.rm = T)) |>
    select(fips:state, one_dose_5plus_pct, one_dose_z)

# A tibble: 3,114 × 5
   fips  county           state one_dose_5plus_pct one_dose_z
   <chr> <chr>            <chr>              <dbl>      <dbl>
 1 26039 Crawford County  MI                  55.7     -0.508
 2 40015 Caddo County     OK                  83.3      1.40
 3 17007 Boone County     IL                  71.1      0.556
 4 12055 Highlands County FL                  68.9      0.404
 5 34029 Ocean County     NJ                  71        0.549
 6 01067 Henry County     AL                  58.5     -0.314
 7 27037 Dakota County    MN                  81        1.24
 8 27115 Pine County      MN                  56.5     -0.452
 9 51750 Radford city     VA                  41.5     -1.49
10 22009 Avoyelles Parish LA                  59.7     -0.231
# ℹ 3,104 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# Where Does 60% Stand?

- The z-score of 60% in each age group, and the share of counties below 60%:

```
> covid_data |>
    summarize(
      z_5plus = (60 - mean(one_dose_5plus_pct, na.rm = T)) / sd(one_dose_5plus_pct, na.rm = T),
      z_65plus = (60 - mean(one_dose_65plus_pct, na.rm = T)) / sd(one_dose_65plus_pct, na.rm = T),
      # TRUE counts as 1 and FALSE as 0, so the mean is the share of counties below 60%
      below_5plus = mean(one_dose_5plus_pct < 60, na.rm = T),
      below_65plus = mean(one_dose_65plus_pct < 60, na.rm = T)
    )

# A tibble: 1 × 4
  z_5plus z_65plus below_5plus below_65plus
    <dbl>    <dbl>       <dbl>        <dbl>
1  -0.211    -2.46       0.455       0.0226
```

- Ages 5+: 60% is $z = -0.21$, a typical county (45% of counties are lower).
- Ages 65+: 60% is $z = -2.46$, near the bottom (only 2% of counties are lower).
    - The shares are counted from the data, not read off a normal (bell-shaped) curve: the 65+ distribution piles up at the 95% cap.

# 2/ Correlation

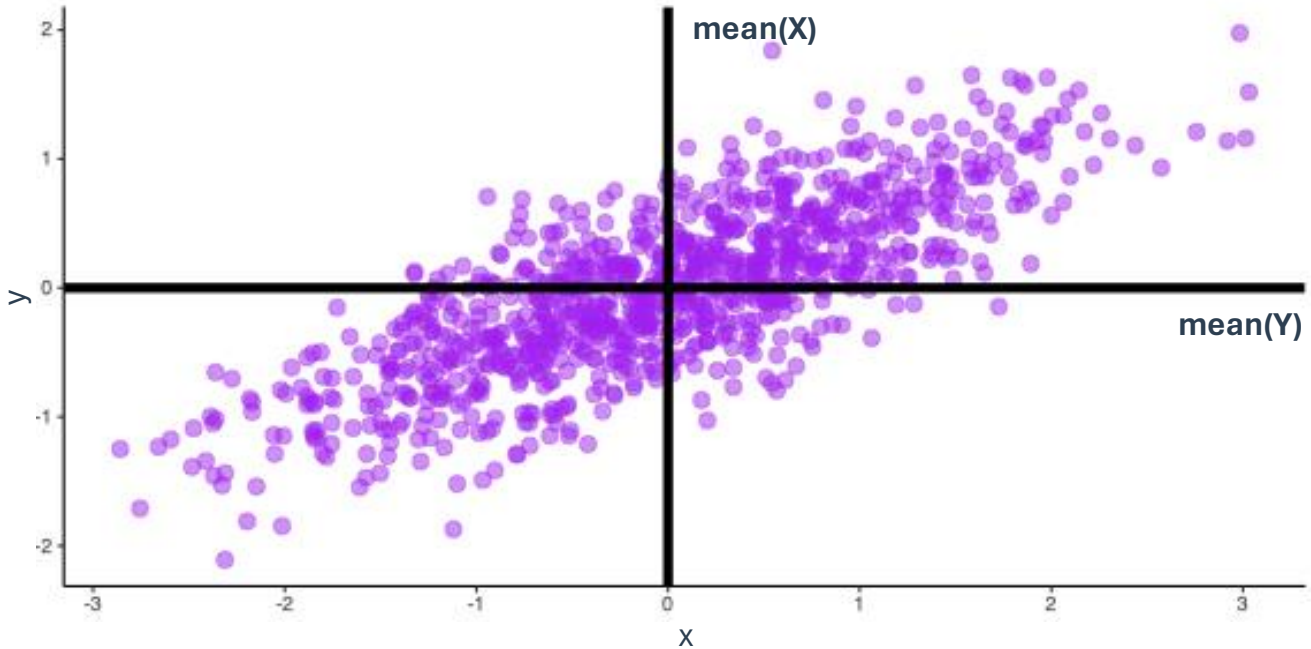
# Correlation

- How do variables move together on average?
- When $x_i$ is big, what is $y_i$ likely to be?
  - Positive correlation: when $x_i$ is big, $y_i$ is also big.
  - Negative correlation: when $x_i$ is big, $y_i$ is small.
  - High magnitude of correlation: data cluster tightly around a line.
- The technical definition of the correlation coefficient:

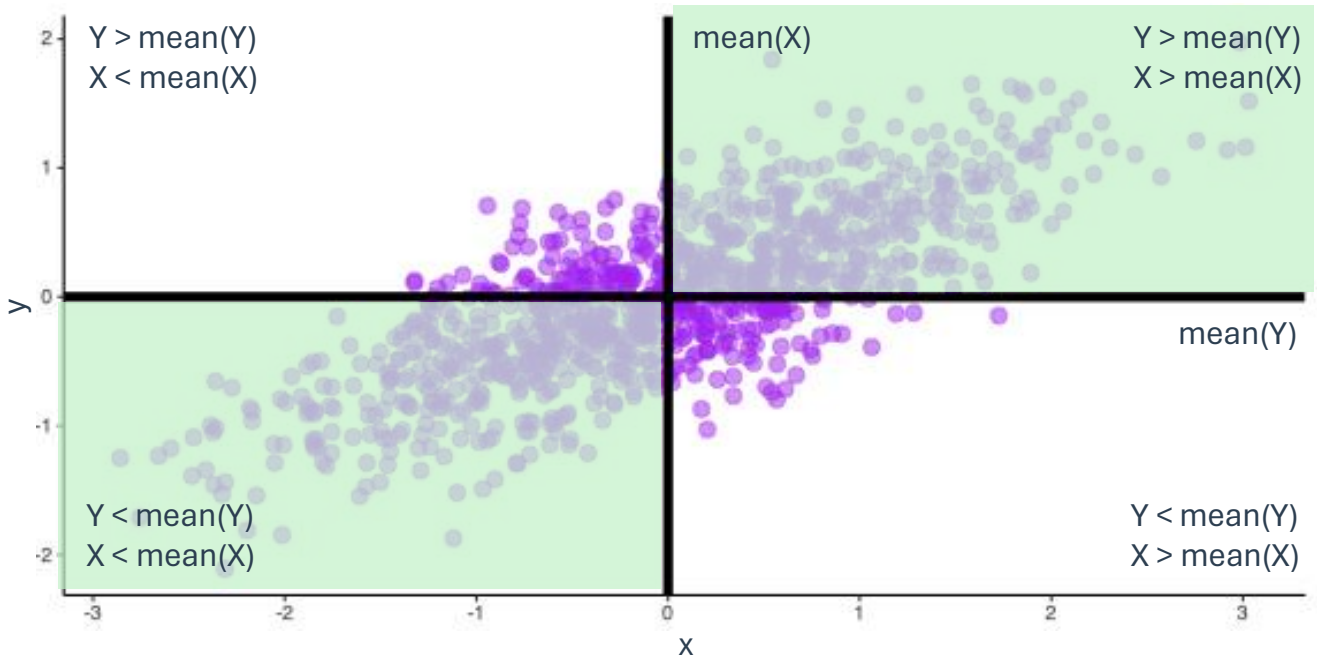
$$\frac{1}{n-1}\sum_{i=1}^{n}\left[(\text{z-score for } x_i)\times(\text{z-score for } y_i)\right]$$

- Interpretation:
  - Correlation is between -1 and 1.
  - Correlation of 0 means no linear association.
  - Positive correlations $\leftrightarrow$ positive associations.
  - Negative correlations $\leftrightarrow$ negative associations.
  - Closer to -1 or 1 means stronger association.

# Correlation Intuition



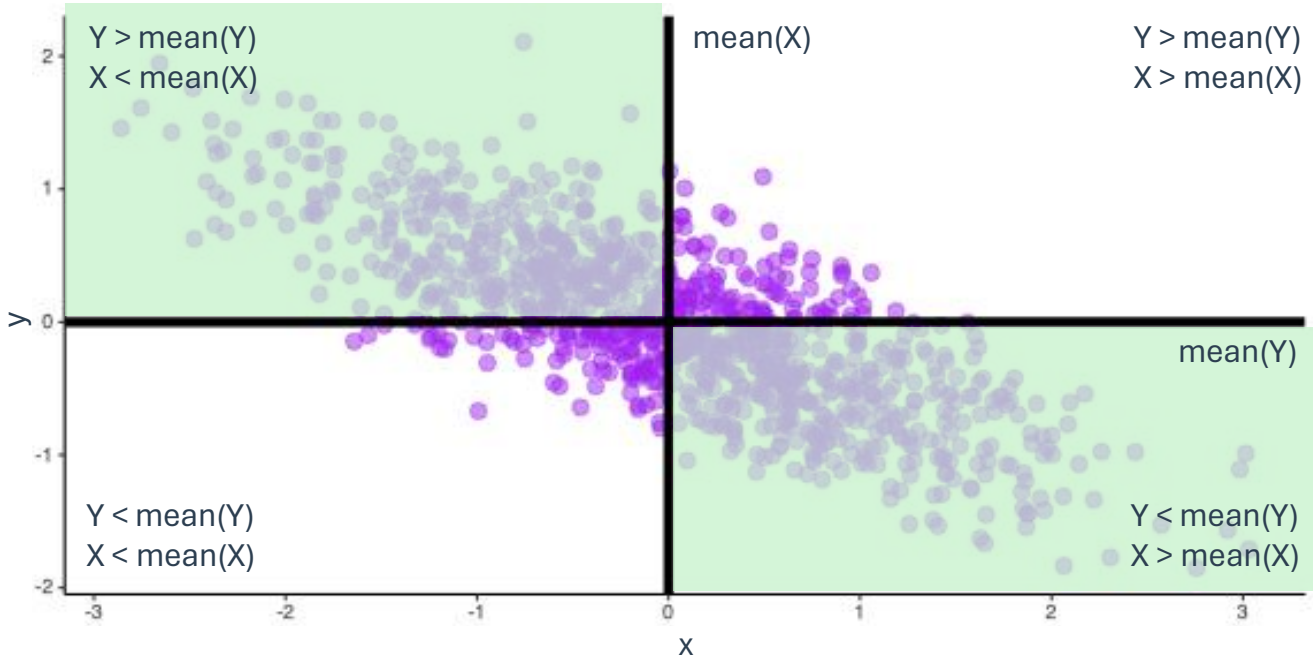
# Correlation Intuition




- Large values of X tend to occur with large values of Y:
  - (z-score for $x_i$) $\times$ (z-score for $y_i$) = (pos. num.) $\times$ (pos. num.) = +
- Small values of X tend to occur with small values of Y:
  - (z-score for $x_i$) $\times$ (z-score for $y_i$) = (neg. num.) $\times$ (neg. num.) = +

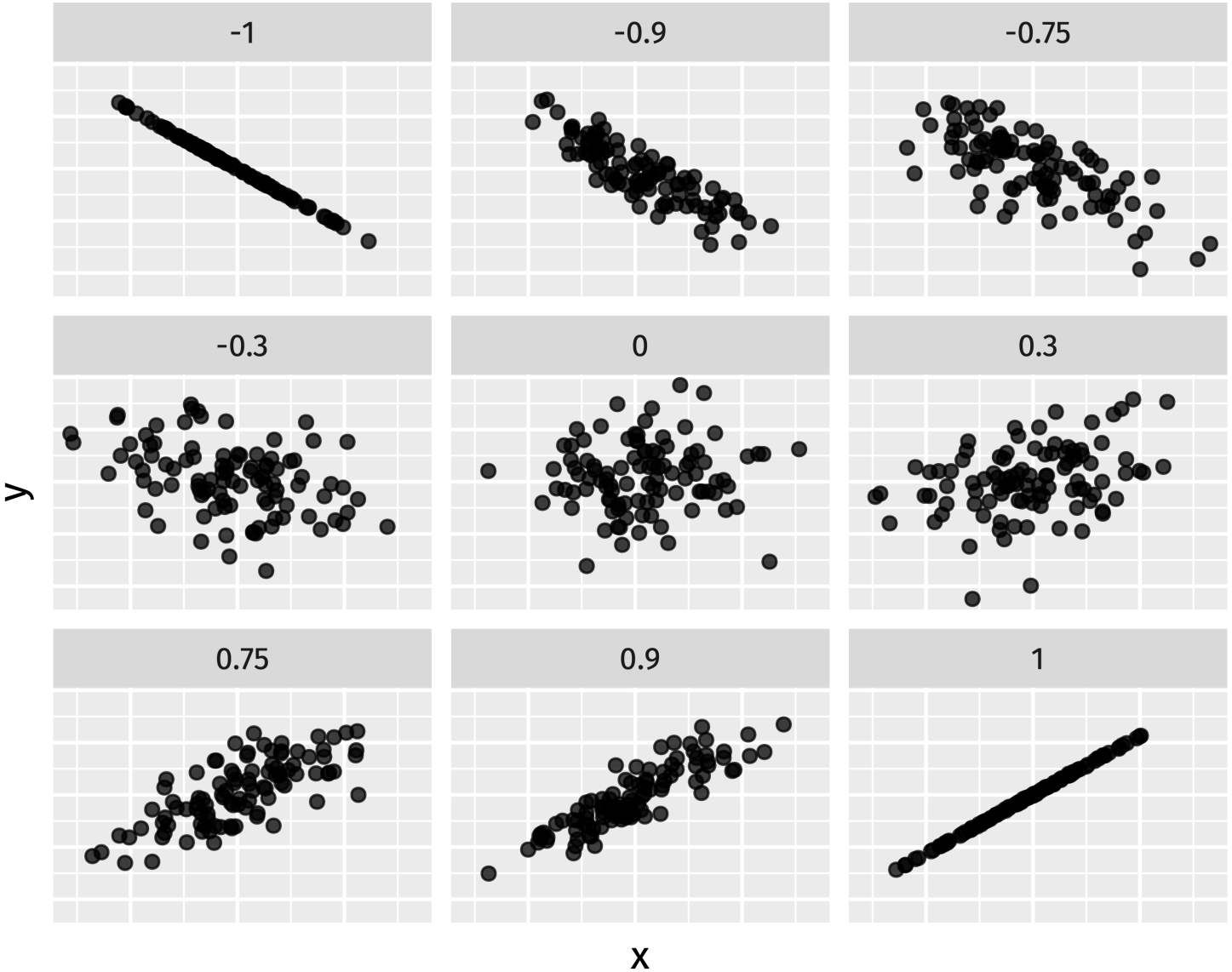
- If these dominate $\longrightarrow$ `positive correlation`

# Correlation Intuition




- Large values of X tend to occur with small values of Y:
  - (z-score for $x_i$) $\times$ (z-score for $y_i$) = (pos. num.) $\times$ (neg. num.) = –
- Small values of X tend to occur with large values of Y:
  - (z-score for $x_i$) $\times$ (z-score for $y_i$) = (neg. num.) $\times$ (pos. num.) = –
- If these dominate $\longrightarrow$ `negative correlation`

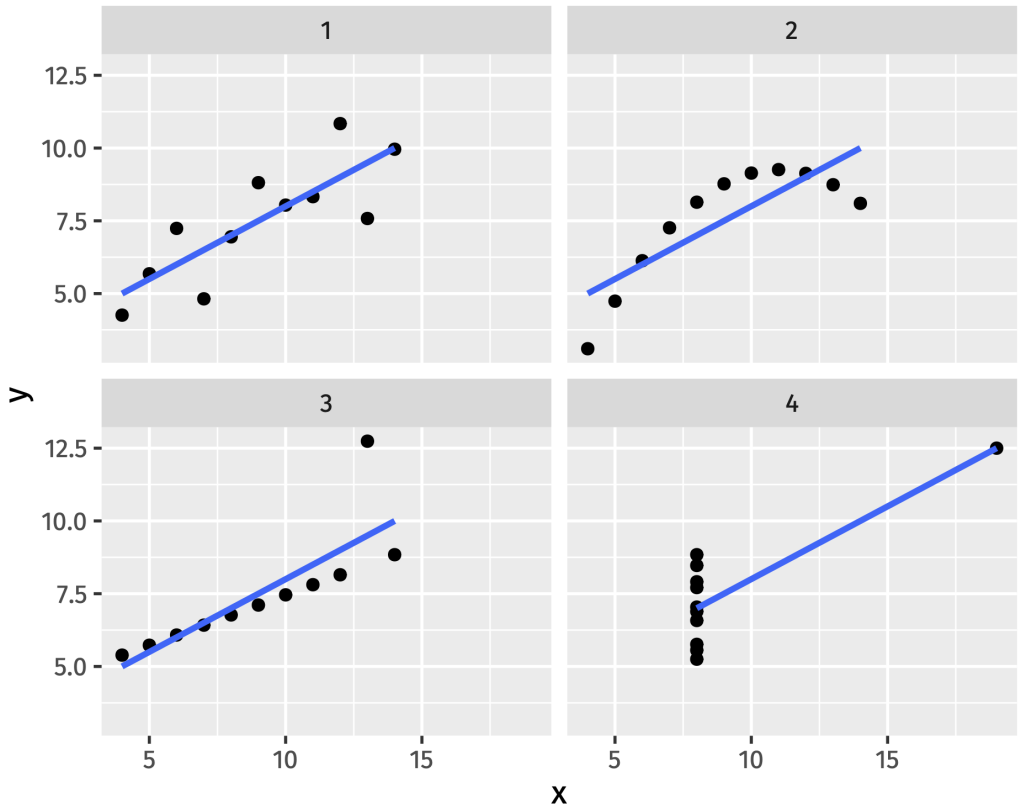
# Correlation Examples



# Properties of Correlation Coefficient

- Correlation measures **linear** association.
- Order doesn't matter: `cor(x,y)` = `cor(y,x)`
- Not affected by changes of scale:
    - `cor(x,y)` = `cor(ax+b, cy+d)` when $a$ and $c$ are both positive/negative
    - Celsius vs. Fahrenheit; dollars vs. pesos; cm vs. in.

# All 4 Relationships have .816 Correlation




- Francis Anscombe's quartet with identical summary stats:
  - Plot 2: inadequate to capture non-linear relationships
  - Plot 3: sensitive to outlier(s)
  - Plot 4: a single influential point creates the correlation by itself; without it, every x is 8 and the correlation is undefined

# Correlations in R

- Use the `cor()` function:

```
> covid_data |>
    summarize(cor(one_dose_5plus_pct, dem_pct_2020))

# A tibble: 1 × 1
  `cor(one_dose_5plus_pct, dem_pct_2020)`
                                    <dbl>
1                                      NA
```

- How should we deal with the missing values?
  - Set the `use = "pairwise"` ⇝ available case analysis (i.e., we will only use units that have values for both columns.)

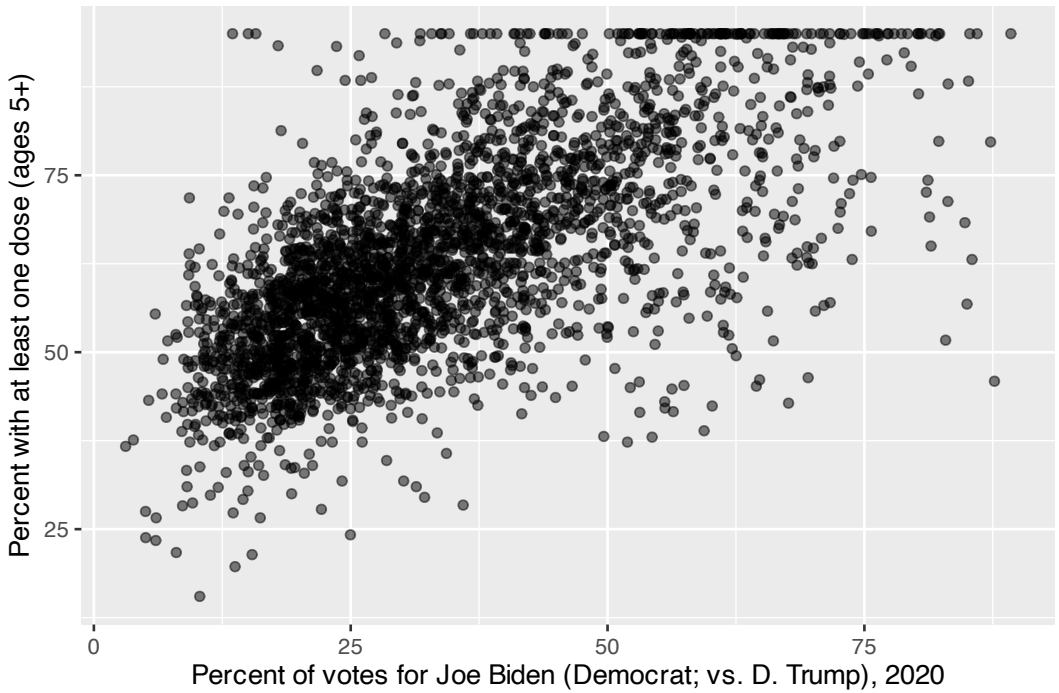
```
> covid_data |>
    summarize(cor(one_dose_5plus_pct, dem_pct_2020, use = "pairwise"))

# A tibble: 1 × 1
  `cor(one_dose_5plus_pct, dem_pct_2020, use = "pairwise")`
                                                      <dbl>
1                                                     0.666
```

# Comparing Correlations #1

- Is a county's Democratic vote share related to its vaccination rate? (2020 election)

```
> covid_data |>
    ggplot(aes(x = dem_pct_2020, y = one_dose_5plus_pct)) + geom_point(alpha = .5) +
    labs(x = "Percent of votes for Joe Biden (Democrat; vs. D. Trump), 2020",
         y = "Percent with at least one dose (ages 5+)")
```

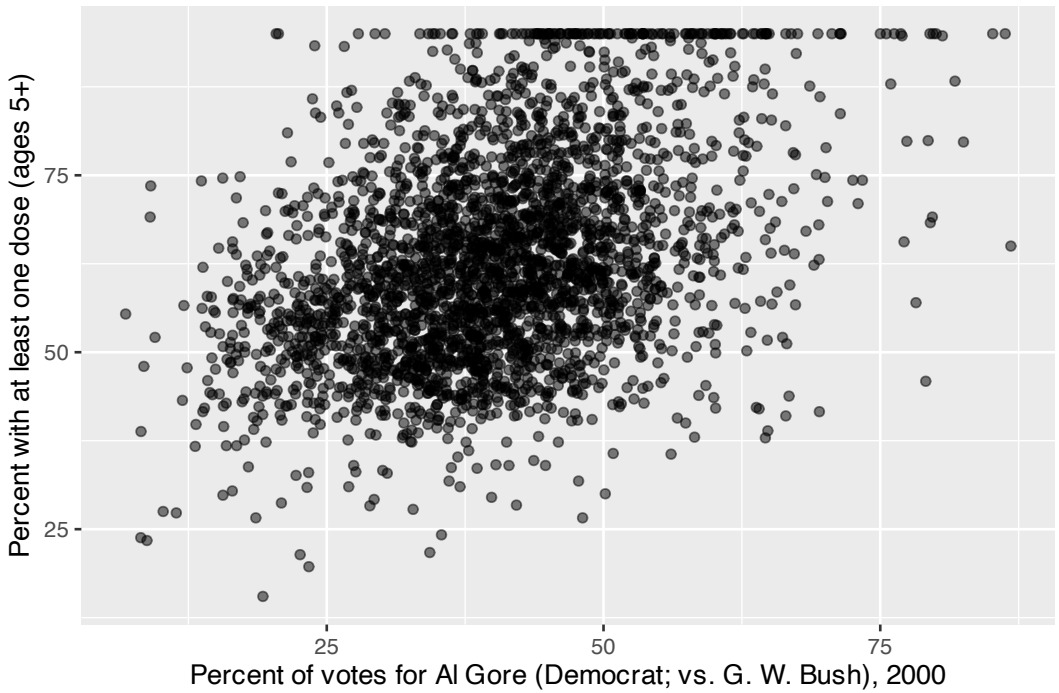



```
> cor(covid_data$one_dose_5plus_pct, covid_data$dem_pct_2020, use = "pairwise")
[1] 0.6664387
```

# Comparing Correlations #2

- Same question with the 2000 election (Gore vs. G. W. Bush):

```
> covid_data |>
    ggplot(aes(x = dem_pct_2000, y = one_dose_5plus_pct)) + geom_point(alpha = .5) +
    labs(x = "Percent of votes for Al Gore (Democrat; vs. G. W. Bush), 2000",
         y = "Percent with at least one dose (ages 5+)")
```

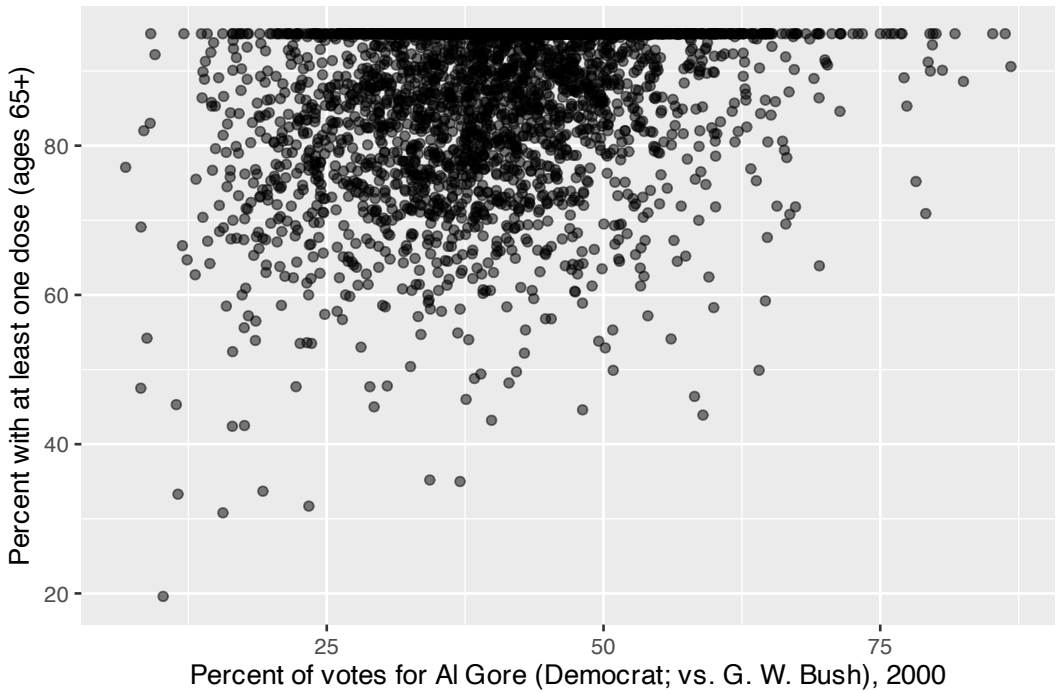



```
> cor(covid_data$one_dose_5plus_pct, covid_data$dem_pct_2000, use = "pairwise")
[1] 0.3941203
```

# Comparing Correlations #3

- Same question for residents aged 65+ (2000 election):

```
> covid_data |>
    ggplot(aes(x = dem_pct_2000, y = one_dose_65plus_pct)) + geom_point(alpha = .5) +
    labs(x = "Percent of votes for Al Gore (Democrat; vs. G. W. Bush), 2000",
         y = "Percent with at least one dose (ages 65+)")
```




```
> cor(covid_data$one_dose_65plus_pct, covid_data$dem_pct_2000, use = "pairwise")
[1] 0.2631949
```

# 3/ Writing our own functions

# Why Write Functions?

- Suppose we wanted to standardize all the variables in our dataset:
  - Copy-pasting code is tedious and prone to failure!

```
> covid_data |>
    mutate(
      one_dose_5p_z =
        (one_dose_5plus_pct - mean(one_dose_5plus_pct, na.rm = TRUE)) /
        sd(one_dose_5plus_pct, na.rm = TRUE),
      one_dose_65p_z =
        (one_dose_65plus_pct - mean(one_dose_65plus_pct, na.rm = TRUE)) /
        sd(one_dose_65plus_pct, na.rm = TRUE),
      booster_z =
        (booster_5plus_pct - mean(booster_5plus_pct, na.rm = TRUE)) /
        sd(booster_5plus_pct, na.rm = TRUE),
      dem_pct_2000_z =
        (dem_pct_2000 - mean(dem_pct_2000, na.rm = TRUE)) /
        sd(dem_pct_2000, na.rm = TRUE),
      dem_pct_2020_z =
        (dem_pct_2020 - mean(dem_pct_2020, na.rm = TRUE)) /
        sd(dem_pct_2020, na.rm = TRUE)
    )
```

# Writing a New Function

- Notice that all of the mutations follow the same template/pattern:

```
> (▬▬ - mean(▬▬, na.rm = TRUE)) / sd(▬▬, na.rm = TRUE)
```

- Only one thing varies: the column of data, represented with ▬ !

# Components of a Function *Redux*

- We can create functions with the following structure:
  - See the R Basics reference materials for additional examples of user-defined functions.

```
name <- function(arguments) {
  body
}
```

- Three components:
  1. **Name**: the name of the function that we'll use to call it. Maybe `z_score`?
  2. **Arguments**: things that we want to vary across calls of our function. We'll use `x`.
  3. **Body**: the code that the function performs.

# Create fn. z_score()!

- Convert our template to a function:

```
> z_score <- function(x) {
    (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
  }
```

- Check if it works:

```
> z_score(c(1,2,3,4,5))
[1] -1.2649111 -0.6324555  0.0000000  0.6324555  1.2649111
```

# Let's Clean Up the Mess

```
> covid_data |>
    mutate(
      one_dose_5p_z = z_score(one_dose_5plus_pct),
      one_dose_65p_z = z_score(one_dose_65plus_pct),
      booster_z = z_score(booster_5plus_pct),
      dem_pct_2000_z = z_score(dem_pct_2000),
      dem_pct_2020_z = z_score(dem_pct_2020)
    )
```

# across() Function

- If we want to replace our variables with z-scores, we can use the across() function to perform many mutations at once:

```
> covid_data |>
    mutate(across(one_dose_5plus_pct:dem_pct_2020, z_score))

# A tibble: 3,114 × 8
   fips  county           state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
   <chr> <chr>            <chr>              <dbl>               <dbl>             <dbl>
 1 26039 Crawford County  MI                -0.508              -0.829             0.531
 2 40015 Caddo County     OK                 1.40                0.843             0.439
 3 17007 Boone County     IL                 0.556               0.795             0.927
 4 12055 Highlands County FL                 0.404               0.720            -0.135
 5 34029 Ocean County     NJ                 0.549               0.843             0.623
 6 01067 Henry County     AL                -0.314              -0.0545           -0.799
 7 27037 Dakota County    MN                 1.24                0.843             2.40
 8 27115 Pine County      MN                -0.452              -0.102             0.577
 9 51750 Radford city     VA                -1.49               -1.16             -2.47
10 22009 Avoyelles Parish LA                -0.231              -0.564            -0.424
# ℹ 3,104 more rows
# ℹ 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
# ℹ Use `print(n = ...)` to see more rows
```

# Alternative Approach #1

- Recall, we could also target all the numeric variables:

```
> covid_data |>
    mutate(across(where(is.numeric), z_score))

# A tibble: 3,114 × 8
   fips  county           state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
   <chr> <chr>            <chr>              <dbl>               <dbl>             <dbl>
 1 26039 Crawford County  MI                -0.508              -0.829             0.531
 2 40015 Caddo County     OK                 1.40                0.843             0.439
 3 17007 Boone County     IL                 0.556               0.795             0.927
 4 12055 Highlands County FL                 0.404               0.720            -0.135
 5 34029 Ocean County     NJ                 0.549               0.843             0.623
 6 01067 Henry County     AL                -0.314              -0.0545           -0.799
 7 27037 Dakota County    MN                 1.24                0.843             2.40
 8 27115 Pine County      MN                -0.452              -0.102             0.577
 9 51750 Radford city     VA                -1.49               -1.16             -2.47
10 22009 Avoyelles Parish LA                -0.231              -0.564            -0.424
# ℹ 3,104 more rows
# ℹ 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
# ℹ Use `print(n = ...)` to see more rows
```

# Alternative Approach #2

- We could also target only the first dose variables by pointing to variable names with a recurring pattern:

```
> covid_data |>
    mutate(across(starts_with("one_dose"), z_score))

# A tibble: 3,114 × 8
   fips  county           state one_dose_5plus_pct one_dose_65plus_pct booster_5plus_pct
   <chr> <chr>            <chr>              <dbl>               <dbl>             <dbl>
 1 26039 Crawford County  MI                -0.508              -0.829              31.2
 2 40015 Caddo County     OK                 1.40                0.843              30.3
 3 17007 Boone County     IL                 0.556               0.795              35.1
 4 12055 Highlands County FL                 0.404               0.720              24.7
 5 34029 Ocean County     NJ                 0.549               0.843              32.1
 6 01067 Henry County     AL                -0.314              -0.0545             18.2
 7 27037 Dakota County    MN                 1.24                0.843              49.5
 8 27115 Pine County      MN                -0.452              -0.102              31.7
 9 51750 Radford city     VA                -1.49               -1.16                1.79
10 22009 Avoyelles Parish LA                -0.231              -0.564              21.9
# ℹ 3,104 more rows
# ℹ 2 more variables: dem_pct_2000 <dbl>, dem_pct_2020 <dbl>
# ℹ Use `print(n = ...)` to see more rows
```

# Adding Arguments to Our Functions

- What if we want to be able to control `na.rm` in the calls to `mean()` and `sd()` in our `z_score()` function?
  - ⇝ Add an argument, with the same default as `mean()` and `sd()`: `na.rm = FALSE`.

```
> z_score2 <- function(x, na.rm = FALSE) {
    (x - mean(x, na.rm = na.rm)) / sd(x, na.rm = na.rm)
  }
```

- If we call the function without specifying `na.rm`?

```
> head(z_score2(covid_data$one_dose_5plus_pct))
[1] NA NA NA NA NA NA
```

  - 112 of 3,114 counties are missing, so `mean()` and `sd()` return NA, and so does every z-score.

- What about now?

```
> head(z_score2(covid_data$one_dose_5plus_pct, na.rm = T))
[1] -0.5076545  1.3982427  0.5557809  0.4038615  0.5488754 -0.3143026
```

# 4/ Pivoting data longer

# Mortality Data

- Data from World Bank on child mortality (under 5 yo) per 1,000 live births in different countries:

```
# Make sure to use read_csv() from tidyverse & readr
> mortality <- read_csv(url("https://bit.ly/3QCCPvc")); mortality

# A tibble: 217 × 52
   country       country_code indicator `1972` `1973` `1974` `1975` `1976` `1977` `1978`
   <chr>         <chr>        <chr>      <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
 1 Aruba         ABW          Mortalit.   NA     NA     NA     NA     NA     NA     NA
 2 Afghanistan   AFG          Mortalit.  291    285.   280.   274.   268    262.   256
 3 Angola        AGO          Mortalit.   NA     NA     NA     NA     NA     NA     NA
 4 Albania       ALB          Mortalit.   NA     NA     NA     NA     NA     NA     97.2
 5 Andorra       AND          Mortalit.   NA     NA     NA     NA     NA     NA     NA
 6 United Arab . ARE          Mortalit.   80.1   72.6   65.7   59.4   53.6   48.3   43.5
 7 Argentina     ARG          Mortalit.   69.7   68.2   66.1   63.3   59.8   55.7   51.2
 8 Armenia       ARM          Mortalit.   NA     NA     NA     NA     87.1   83.6   80.2
 9 American Sam. ASM          Mortalit.   NA     NA     NA     NA     NA     NA     NA
10 Antigua and . ATG          Mortalit.   26.9   25.1   23.5   22.1   20.8   19.5   18.3
# ℹ 207 more rows
# ℹ 42 more variables: `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>,
#   `1983` <dbl>, `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
#   `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>, `1994` <dbl>,
#   `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>, `1999` <dbl>, `2000` <dbl>,
#   `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, `2004` <dbl>, `2005` <dbl>, `2006` <dbl>,
#   `2007` <dbl>, `2008` <dbl>, `2009` <dbl>, `2010` <dbl>, `2011` <dbl>, . . .
# ℹ Use `print(n = ...)` to see more rows
```

# Pivoting Longer

- Mortality data is in a “wide” format (years in columns).
  - We can convert this to country-year rows with `pivot_longer()`.

- Syntax:

```
mydata |>
  pivot_longer(
    cols = <<variables to pivot>>,
    names_to = <<new variable to put column names>>,
    values_to = <<new variable to put column values>>
)
```

# Pivoting the Mortality Data

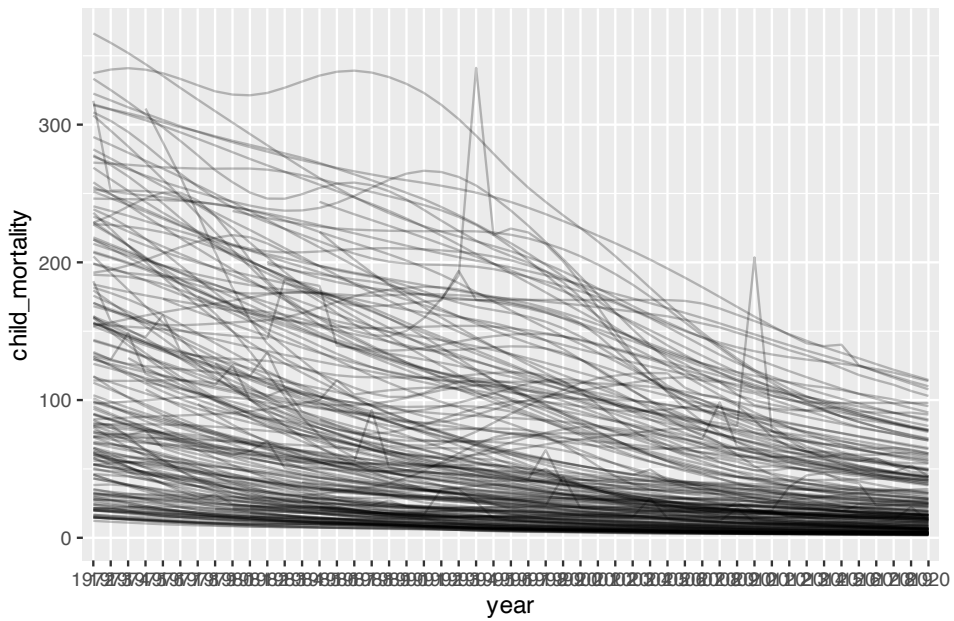
- The 'long' data contains the years in one column `year` with corresponding mortality rates in `child_mortality`:

```
> mortality |>
    select(-indicator) |>
    pivot_longer(
      cols = `1972`:`2020`,
      names_to = "year",
      values_to = "child_mortality"
    )

# A tibble: 10,633 × 4
   country country_code year  child_mortality
   <chr>   <chr>        <chr>           <dbl>
 1 Aruba   ABW          1972               NA
 2 Aruba   ABW          1973               NA
 3 Aruba   ABW          1974               NA
 4 Aruba   ABW          1975               NA
 5 Aruba   ABW          1976               NA
 6 Aruba   ABW          1977               NA
 7 Aruba   ABW          1978               NA
 8 Aruba   ABW          1979               NA
 9 Aruba   ABW          1980               NA
10 Aruba   ABW          1981               NA
# ℹ 10,623 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# Let's Do Line Plots!

```
> mortality |>
    select(-indicator) |>
    pivot_longer(
      cols = `1972`:`2020`,
      names_to = "year",
      values_to = "child_mortality"
    ) |>
    ggplot(mapping = aes(x = year, y = child_mortality, group = country)) +
    geom_line(alpha = 0.25)
```




- What's going on with the x-axis label?

# Making Sure Year is Numeric

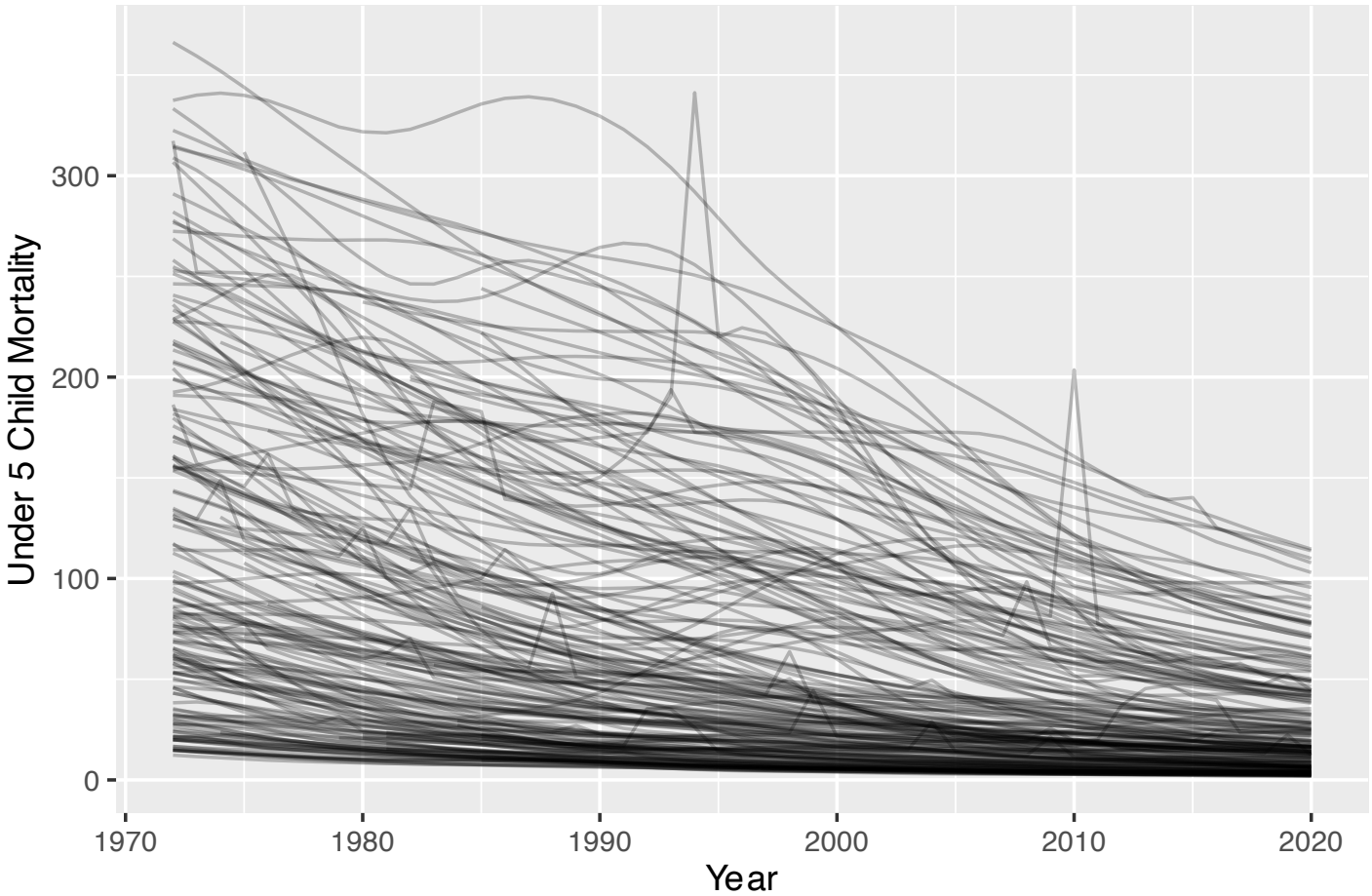
- By default, pivoted column names are characters, but we can transform them:

```
> mortality_long <- mortality |>
    select(-indicator) |>
    pivot_longer(
      cols = `1972`:`2020`,
      names_to = "year",
      values_to = "child_mortality"
    ) |>
    mutate(year = as.integer(year)); mortality_long

# A tibble: 10,633 × 4
   country country_code  year child_mortality
   <chr>   <chr>        <int>           <dbl>
 1 Aruba   ABW           1972              NA
 2 Aruba   ABW           1973              NA
 3 Aruba   ABW           1974              NA
 4 Aruba   ABW           1975              NA
 5 Aruba   ABW           1976              NA
 6 Aruba   ABW           1977              NA
 7 Aruba   ABW           1978              NA
 8 Aruba   ABW           1979              NA
 9 Aruba   ABW           1980              NA
10 Aruba   ABW           1981              NA
# ℹ 10,623 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# Let's (Re)Do Line Plots!

```
> mortality_long |>
    ggplot(mapping = aes(x = year, y = child_mortality, group = country)) +
    geom_line(alpha = 0.25) +
    labs(x = "Year", y = "Under 5 Child Mortality")
```



# Spotify Data

- Data on ranks of 50 most popular songs streamed by Spotify users in the US, during 2020 (Spotify Charts):

```
# Make sure to use read_csv() from tidyverse & readr
> spotify <- read_csv(url("https://bit.ly/49tn2qR")); spotify

# A tibble: 490 × 54
   `Track Name`  Artist week1 week2 week3 week4 week5 week6 week7 week8 week9 week10
   <chr>         <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
 1 The Box       Roddy.     1     1     1     1     1     1     1     1     1      1
 2 ROXANNE       Arizo.     2     4     5     4     4     4     6     7     9     21
 3 Yummy         Justi.     3     6    17    17    17    24    15    32    NA     NA
 4 Circles       Post .     4     7     9    10     7    10    11    10    17     26
 5 BOP           DaBaby     5     5     7     5    11    12    18    18    32     47
 6 Falling       Trevo.     6     8    10     7     6     8    10    11    18     28
 7 Dance Monkey  Tones.     7    13    13    12    12    13    17    13    21     33
 8 Bandit (with. Juice.     8    11    14    14    15    20    27    26    42     NA
 9 Futsal Shuff. Lil U.     9     9    19    21    24    32    40    49    NA     NA
10 everything i. Billi.    10    17    28     9     8    11    14    17    29     NA
# ℹ 480 more rows
# ℹ 42 more variables: week11 <dbl>, week12 <dbl>, week13 <dbl>, week14 <dbl>,
#   week15 <dbl>, week16 <dbl>, week17 <dbl>, week18 <dbl>, week19 <dbl>,
#   week20 <dbl>, week21 <dbl>, week22 <dbl>, week23 <dbl>, week24 <dbl>,
#   week25 <dbl>, week26 <dbl>, week27 <dbl>, week28 <dbl>, week29 <dbl>,
#   week30 <dbl>, week31 <dbl>, week32 <dbl>, week33 <dbl>, week34 <dbl>,
#   week35 <dbl>, week36 <dbl>, week37 <dbl>, week38 <dbl>, week39 <dbl>,
# ℹ Use `print(n = ...)` to see more rows
```

# Pivoting Not Ideal

- Our previous approach isn't ideal because of the week prefix:
  - week_of_year will be a character vector which won't be ideal when we visualize/analyze the data & require additional steps!

```
> spotify |>
    pivot_longer(
      cols = c(-`Track Name`, -Artist),
      names_to = "week_of_year",
      values_to = "rank"
    )

# A tibble: 25,480 × 4
   `Track Name` Artist      week_of_year  rank
   <chr>        <chr>       <chr>        <dbl>
 1 The Box      Roddy Ricch week1            1
 2 The Box      Roddy Ricch week2            1
 3 The Box      Roddy Ricch week3            1
 4 The Box      Roddy Ricch week4            1
 5 The Box      Roddy Ricch week5            1
 6 The Box      Roddy Ricch week6            1
 7 The Box      Roddy Ricch week7            1
 8 The Box      Roddy Ricch week8            1
 9 The Box      Roddy Ricch week9            1
10 The Box      Roddy Ricch week10           1
# ℹ 25,470 more rows
# ℹ Use `print(n = ...)` to see more rows
```

  - Q: What to do?

# Removing a Column Name Prefix

- When the data in the column name has a fixed prefix, we can use the `names_prefix` argument to remove it when moving the data to rows

```
> spotify |>
    pivot_longer(
      cols = c(-`Track Name`, -Artist),
      names_to = "week_of_year",
      values_to = "rank",
      names_prefix = "week"
    ) |> mutate(
      week_of_year = as.integer(week_of_year)
    )

# A tibble: 25,480 × 4
   `Track Name` Artist      week_of_year  rank
   <chr>        <chr>              <int> <dbl>
 1 The Box      Roddy Ricch            1     1
 2 The Box      Roddy Ricch            2     1
 3 The Box      Roddy Ricch            3     1
 4 The Box      Roddy Ricch            4     1
 5 The Box      Roddy Ricch            5     1
 6 The Box      Roddy Ricch            6     1
 7 The Box      Roddy Ricch            7     1
 8 The Box      Roddy Ricch            8     1
 9 The Box      Roddy Ricch            9     1
10 The Box      Roddy Ricch           10     1
# ℹ 25,470 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# 5/ Joining data sets

# Revisiting Mortality Data

- Data from World Bank on child mortality (under 5 yo) per 1,000 live births in different countries:

```
# Make sure to use read_csv() from tidyverse & readr
> mortality <- read_csv(url("https://bit.ly/3QCCPvc")); mortality

# A tibble: 217 × 52
   country       country_code indicator `1972` `1973` `1974` `1975` `1976` `1977` `1978`
   <chr>         <chr>        <chr>      <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
 1 Aruba         ABW          Mortalit.   NA     NA     NA     NA     NA     NA     NA
 2 Afghanistan   AFG          Mortalit.  291    285.   280.   274.   268    262.   256
 3 Angola        AGO          Mortalit.   NA     NA     NA     NA     NA     NA     NA
 4 Albania       ALB          Mortalit.   NA     NA     NA     NA     NA     NA     97.2
 5 Andorra       AND          Mortalit.   NA     NA     NA     NA     NA     NA     NA
 6 United Arab . ARE          Mortalit.   80.1   72.6   65.7   59.4   53.6   48.3   43.5
 7 Argentina     ARG          Mortalit.   69.7   68.2   66.1   63.3   59.8   55.7   51.2
 8 Armenia       ARM          Mortalit.   NA     NA     NA     NA     87.1   83.6   80.2
 9 American Sam. ASM          Mortalit.   NA     NA     NA     NA     NA     NA     NA
10 Antigua and . ATG          Mortalit.   26.9   25.1   23.5   22.1   20.8   19.5   18.3
# ℹ 207 more rows
# ℹ 42 more variables: `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>,
#   `1983` <dbl>, `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
#   `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>, `1994` <dbl>,
#   `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>, `1999` <dbl>, `2000` <dbl>,
#   `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, `2004` <dbl>, `2005` <dbl>, `2006` <dbl>,
#   `2007` <dbl>, `2008` <dbl>, `2009` <dbl>, `2010` <dbl>, `2011` <dbl>, . . .
# ℹ Use `print(n = ...)` to see more rows
```

# Gapminder Data

- Recall we have access to life expectancy, population size, and income level (i.e., GDP per Cap) at the country level!
  - We may want to enrich our analysis with this additional information.

```
> require(gapminder)
> gapminder

# A tibble: 1,704 × 6
   country     continent  year lifeExp      pop gdpPercap
   <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
 1 Afghanistan Asia       1952    28.8  8425333      779.
 2 Afghanistan Asia       1957    30.3  9240934      821.
 3 Afghanistan Asia       1962    32.0 10267083      853.
 4 Afghanistan Asia       1967    34.0 11537966      836.
 5 Afghanistan Asia       1972    36.1 13079460      740.
 6 Afghanistan Asia       1977    38.4 14880372      786.
 7 Afghanistan Asia       1982    39.9 12881816      978.
 8 Afghanistan Asia       1987    40.8 13867957      852.
 9 Afghanistan Asia       1992    41.7 16317921      649.
10 Afghanistan Asia       1997    41.8 22227415      635.
# ℹ 1,694 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# Joining Data Sets

- What if we want to add the `child_mortality` variable to the `gapminder` data?
- Just add the columns (e.g. `cbind()`)?
  - Issue: rows are not aligned properly!

```
> gapminder |>
    select(country, year) |>
    head(10)

# A tibble: 10 × 2
   country      year
   <fct>       <int>
 1 Afghanistan  1952
 2 Afghanistan  1957
 3 Afghanistan  1962
 4 Afghanistan  1967
 5 Afghanistan  1972
 6 Afghanistan  1977
 7 Afghanistan  1982
 8 Afghanistan  1987
 9 Afghanistan  1992
10 Afghanistan  1997
```

```
> mortality_long |>
    select(country, year) |>
    head(10)

# A tibble: 10 × 2
   country  year
   <chr>   <int>
 1 Aruba    1972
 2 Aruba    1973
 3 Aruba    1974
 4 Aruba    1975
 5 Aruba    1976
 6 Aruba    1977
 7 Aruba    1978
 8 Aruba    1979
 9 Aruba    1980
10 Aruba    1981
```

# Key Variables

- A **primary key** is a variable or set of variables that uniquely identifies rows in the data.
  - $\{$country, year$\}$ in both gapminder and mortality_long: each pair appears only once.

- A **foreign key** is the corresponding variable(s) in another table. Its values can repeat.
  - tailnum in the flights data (one plane, many flights) corresponds to tailnum in the planes data.

- If we 'align' the two tables based on these variables, we can add variables from one to the other (i.e., join them).
  - The table we add variables from must be matched on its primary key; otherwise rows get duplicated.

# Checking if the Keys are Unique

- Joining won't work properly if the key of the table we add variables from is not unique. Let's check.
  - What happens if a key repeats: slide 54.

Checking gapminder:

```
> gapminder |>
    count(country, year) |>
    filter(n > 1)

# A tibble: 0 × 3
# ℹ 3 variables: country <fct>,
#   year <int>, n <int>
```

Checking mortality_long (we add variables from it):

```
> mortality_long |>
    count(country, year) |>
    filter(n > 1)

# A tibble: 0 × 3
# ℹ 3 variables: country <chr>,
#   year <int>, n <int>
```

- No duplicate rows based on the combination of 'country' and 'year' in the datasets.
  - Each pair of 'country' and 'year' appears only once, serving as a unique identifier for each observation.

# `left_join()`: Add Variables to the First Table (x)

- `left_join()` keeps all rows of the first (piped) table, x:

```
> gapminder |>
    left_join(mortality_long) |>
    select(country, year, lifeExp, pop, gdpPercap, child_mortality) |>
    head(n = 6)

Joining with `by = join_by(country, year)`
# A tibble: 6 × 6
  country      year lifeExp      pop gdpPercap child_mortality
  <chr>       <int>   <dbl>    <int>     <dbl>           <dbl>
1 Afghanistan  1952    28.8  8425333      779.             NA
2 Afghanistan  1957    30.3  9240934      821.             NA
3 Afghanistan  1962    32.0 10267083      853.             NA
4 Afghanistan  1967    34.0 11537966      836.             NA
5 Afghanistan  1972    36.1 13079460      740.            291
6 Afghanistan  1977    38.4 14880372      786.            262.
```

- Rows in x that have no match in y?
  - Stored as **NA**!

# inner_join(): add and filter

- inner_join() adds the variables from y and keeps only the rows whose keys are in both tables:

```
> gapminder |>
    inner_join(mortality_long) |>
    select(country, year, lifeExp, pop, gdpPercap, child_mortality) |>
    head(n = 6)

Joining with `by = join_by(country, year)`
# A tibble: 6 × 6
  country      year lifeExp      pop gdpPercap child_mortality
  <chr>       <int>   <dbl>    <int>     <dbl>           <dbl>
1 Afghanistan  1972    36.1 13079460      740.            291
2 Afghanistan  1977    38.4 14880372      786.            262.
3 Afghanistan  1982    39.9 12881816      978.            231.
4 Afghanistan  1987    40.8 13867957      852.            198.
5 Afghanistan  1992    41.7 16317921      649.            166.
6 Afghanistan  1997    41.8 22227415      635.            142.
```

  - R automatically determined the key variables.
    - Ideally, we should specify the joining/key variables by setting by = c("country", "year") in the join fns.

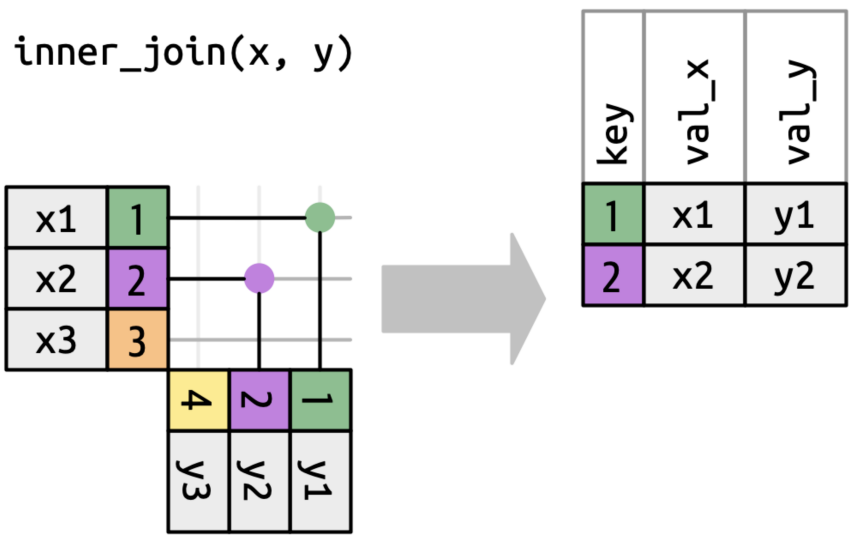
# How Inner Joins Work

- Two data sets[1]:

| x | |
|---|---|
| 1 | x1 |
| 2 | x2 |
| 3 | x3 |

| y | |
|---|---|
| 1 | y1 |
| 2 | y2 |
| 4 | y3 |

- Find matching keys:




[1]Credit: R for Data Science

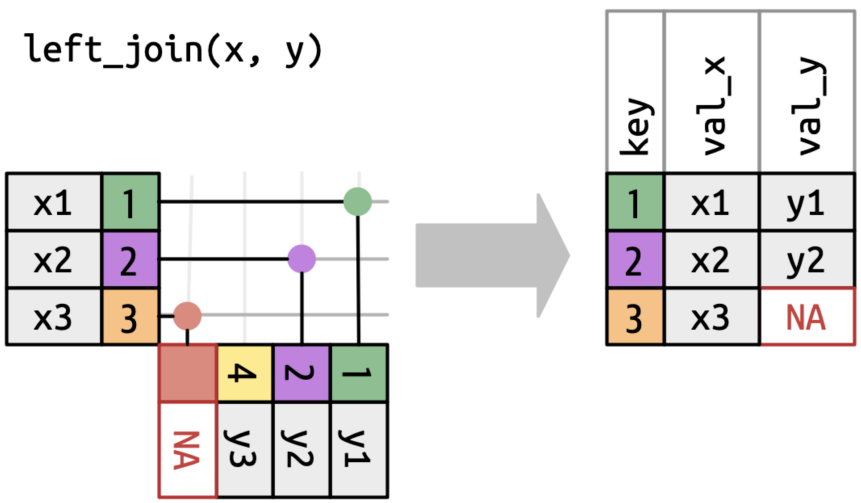
# How Left Joins Work

- Two data sets[2]:

| x | |
|---|---|
| 1 | x1 |
| 2 | x2 |
| 3 | x3 |

| y | |
|---|---|
| 1 | y1 |
| 2 | y2 |
| 4 | y3 |

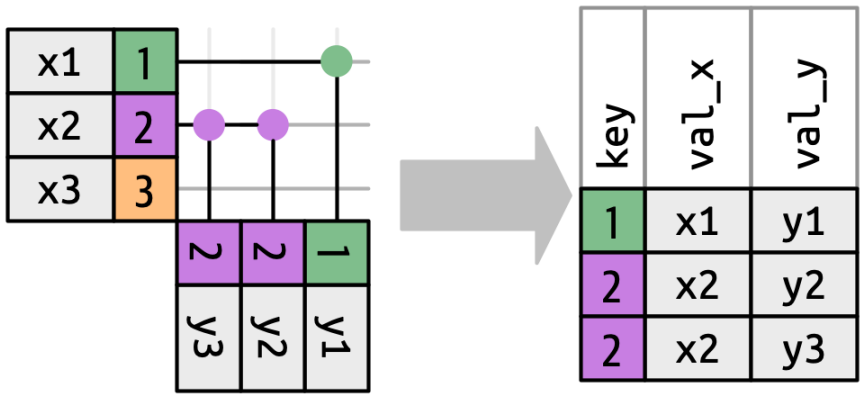
- Keep all x keys (all rows of x):




[2]Credit: R for Data Science

# When a Key Repeats

- So far, each row of x matched zero or one row of y. What happens if a row of x matches more than one row of y?[3]




- A row of x is copied once for each match.
  - In this inner join, key 2 appears twice in y. So x1 matches one row of y, x2 matches two rows, and x3 matches none: the result has x1 once, x2 twice, and no x3.
  - `left_join()` does the same (and keeps x3 with NA, as on slide 53).

[3] Credit: R for Data Science (2e), Section 19.4.1 and Figure 19.9

# More Complicated Example: Flight Data

```
> library(nycflights13)
> flights2 <- flights |>
    select(year, time_hour, origin, dest, tailnum, carrier); flights2

# A tibble: 336,776 × 6
    year time_hour           origin dest  tailnum carrier
   <int> <dttm>              <chr>  <chr> <chr>   <chr>
 1  2013 2013-01-01 05:00:00 EWR    IAH   N14228  UA
 2  2013 2013-01-01 05:00:00 LGA    IAH   N24211  UA
 3  2013 2013-01-01 05:00:00 JFK    MIA   N619AA  AA
 4  2013 2013-01-01 05:00:00 JFK    BQN   N804JB  B6
 5  2013 2013-01-01 06:00:00 LGA    ATL   N668DN  DL
 6  2013 2013-01-01 05:00:00 EWR    ORD   N39463  UA
 7  2013 2013-01-01 06:00:00 EWR    FLL   N516JB  B6
 8  2013 2013-01-01 06:00:00 LGA    IAD   N829AS  EV
 9  2013 2013-01-01 06:00:00 JFK    MCO   N593JB  B6
10  2013 2013-01-01 06:00:00 LGA    ORD   N3ALAA  AA
# ℹ 336,766 more rows
# ℹ Use `print(n = ...)` to see more rows
```

- `nycflights13`: all flights that departed NYC in 2013, plus plane data.

# Planes Data

```
> planes2 <- planes |>
    select(tailnum, year, type, engine, seats); planes2

# A tibble: 3,322 × 5
   tailnum  year type                    engine    seats
   <chr>   <int> <chr>                   <chr>     <int>
 1 N10156   2004 Fixed wing multi engine Turbo-fan    55
 2 N102UW   1998 Fixed wing multi engine Turbo-fan   182
 3 N103US   1999 Fixed wing multi engine Turbo-fan   182
 4 N104UW   1999 Fixed wing multi engine Turbo-fan   182
 5 N10575   2002 Fixed wing multi engine Turbo-fan    55
 6 N105UW   1999 Fixed wing multi engine Turbo-fan   182
 7 N107US   1999 Fixed wing multi engine Turbo-fan   182
 8 N108UW   1999 Fixed wing multi engine Turbo-fan   182
 9 N109UW   1999 Fixed wing multi engine Turbo-fan   182
10 N110UW   1999 Fixed wing multi engine Turbo-fan   182
# ℹ 3,312 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# What Happens with 'Naive' Join?

```
> flights2 |>
    left_join(planes2)

Joining with `by = join_by(year, tailnum)`
# A tibble: 336,776 × 9
    year time_hour           origin dest  tailnum carrier type  engine seats
   <int> <dttm>              <chr>  <chr> <chr>   <chr>   <chr> <chr>  <int>
 1  2013 2013-01-01 05:00:00 EWR    IAH   N14228  UA      NA    NA        NA
 2  2013 2013-01-01 05:00:00 LGA    IAH   N24211  UA      NA    NA        NA
 3  2013 2013-01-01 05:00:00 JFK    MIA   N619AA  AA      NA    NA        NA
 4  2013 2013-01-01 05:00:00 JFK    BQN   N804JB  B6      NA    NA        NA
 5  2013 2013-01-01 06:00:00 LGA    ATL   N668DN  DL      NA    NA        NA
 6  2013 2013-01-01 05:00:00 EWR    ORD   N39463  UA      NA    NA        NA
 7  2013 2013-01-01 06:00:00 EWR    FLL   N516JB  B6      NA    NA        NA
 8  2013 2013-01-01 06:00:00 LGA    IAD   N829AS  EV      NA    NA        NA
 9  2013 2013-01-01 06:00:00 JFK    MCO   N593JB  B6      NA    NA        NA
10  2013 2013-01-01 06:00:00 LGA    ORD   N3ALAA  AA      NA    NA        NA
# ℹ 336,766 more rows
# ℹ Use `print(n = ...)` to see more rows
```

- As usual, *R* automatically detects which columns should be used as key variables ⇝ `c(year, tailnum)` – Wait..
  - `year` in `flights` is flight year vs. the manufacturing year for the airplane in `planes`!

# Specify the Joining Variables

```
> flights2 |>
    left_join(planes2, by = "tailnum")

# A tibble: 336,776 × 10
   year.x time_hour           origin dest  tailnum carrier year.y type  engine seats
    <int> <dttm>              <chr>  <chr> <chr>   <chr>    <int> <chr> <chr>  <int>
 1   2013 2013-01-01 05:00:00 EWR    IAH   N14228  UA        1999 Fixe. Turbo.   149
 2   2013 2013-01-01 05:00:00 LGA    IAH   N24211  UA        1998 Fixe. Turbo.   149
 3   2013 2013-01-01 05:00:00 JFK    MIA   N619AA  AA        1990 Fixe. Turbo.   178
 4   2013 2013-01-01 05:00:00 JFK    BQN   N804JB  B6        2012 Fixe. Turbo.   200
 5   2013 2013-01-01 06:00:00 LGA    ATL   N668DN  DL        1991 Fixe. Turbo.   178
 6   2013 2013-01-01 05:00:00 EWR    ORD   N39463  UA        2012 Fixe. Turbo.   191
 7   2013 2013-01-01 06:00:00 EWR    FLL   N516JB  B6        2000 Fixe. Turbo.   200
 8   2013 2013-01-01 06:00:00 LGA    IAD   N829AS  EV        1998 Fixe. Turbo.    55
 9   2013 2013-01-01 06:00:00 JFK    MCO   N593JB  B6        2004 Fixe. Turbo.   200
10   2013 2013-01-01 06:00:00 LGA    ORD   N3ALAA  AA          NA NA    NA        NA
# ℹ 336,766 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# Or Change Variable Names

```
> flights2 |>
    left_join(planes2 |>
                rename(manufacture_year = year))

Joining with `by = join_by(tailnum)`
# A tibble: 336,776 × 10
    year time_hour           origin dest  tailnum carrier manufacture_year type
   <int> <dttm>              <chr>  <chr> <chr>   <chr>               <int> <chr>
 1  2013 2013-01-01 05:00:00 EWR    IAH   N14228  UA                   1999 Fixed wi.
 2  2013 2013-01-01 05:00:00 LGA    IAH   N24211  UA                   1998 Fixed wi.
 3  2013 2013-01-01 05:00:00 JFK    MIA   N619AA  AA                   1990 Fixed wi.
 4  2013 2013-01-01 05:00:00 JFK    BQN   N804JB  B6                   2012 Fixed wi.
 5  2013 2013-01-01 06:00:00 LGA    ATL   N668DN  DL                   1991 Fixed wi.
 6  2013 2013-01-01 05:00:00 EWR    ORD   N39463  UA                   2012 Fixed wi.
 7  2013 2013-01-01 06:00:00 EWR    FLL   N516JB  B6                   2000 Fixed wi.
 8  2013 2013-01-01 06:00:00 LGA    IAD   N829AS  EV                   1998 Fixed wi.
 9  2013 2013-01-01 06:00:00 JFK    MCO   N593JB  B6                   2004 Fixed wi.
10  2013 2013-01-01 06:00:00 LGA    ORD   N3ALAA  AA                     NA NA
# ℹ 336,766 more rows
# ℹ 2 more variables: engine <chr>, seats <int>
# ℹ Use `print(n = ...)` to see more rows
```

# What Comes Next?

- Today: z-scores and correlation, writing our own functions, pivoting data longer, and joining data sets.
- Next class (Oct 29): **prediction and iteration**. Complete the Required reading in the weekly schedule beforehand.
- Milestone 2 (data and proposal) is due Sunday, Oct 25. The guide to finding data is on the course page.
- Assignment 2 is due before class starts on Thursday, Nov 5.

# Enjoy Your Weekends! :)

*Contact Information:*
jaewon.yoo@iss.nthu.edu.tw
https://j1yoo.github.io/



# Code for Visualization

```
#### Bivariate Relationships and Tidying Data
## Data for the histograms ("Is 60% Vaccinated A Lot?" and the next three slides):
pacman::p_load(tidyverse); covid_data <- read_csv(url("https://bit.ly/3FOLohq"))

# Pivot data longer:
covid_long <- covid_data |>
  pivot_longer(
    cols = c(one_dose_5plus_pct, one_dose_65plus_pct),
    names_to = "population",
    values_to = "one_dose_pct"
  ) |>
  mutate(Population = if_else(population == "one_dose_5plus_pct", "Ages 5+", "Ages 65+"))

## Is 60% Vaccinated A Lot?
ggplot(data = covid_long, aes(x = one_dose_pct, fill = Population)) +
  geom_histogram(alpha = .5) +
  geom_vline(xintercept = 60) +
  labs(x = "Percent of Population with at least One Dose", y = "Count")

## Original Distributions
ggplot(data = covid_long, aes(x = one_dose_pct, fill = Population)) +
  geom_histogram(alpha = .5) +
  geom_vline(aes(xintercept = 60, color = Population)) +
  labs(x = "Percent of Population with at least One Dose", y = "Count")
```

# Code for Visualization Cont.

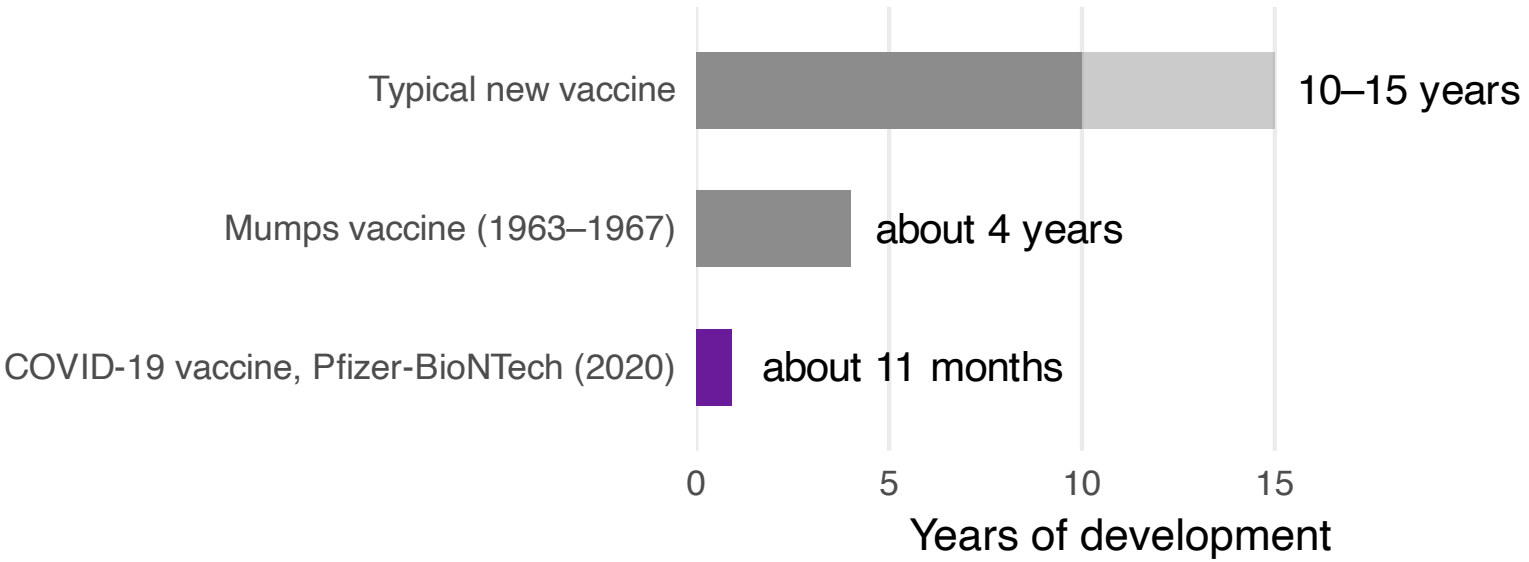
```r
## Centered Distributions
centered_60 <- covid_long |>
  group_by(Population) |>
  summarize(centered_60 = 60 - mean(one_dose_pct, na.rm = T)); centered_60

covid_long |>
  group_by(Population) |>
  mutate(one_dose_centered = one_dose_pct - mean(one_dose_pct, na.rm = T),
         centered_60 = 60 - mean(one_dose_pct, na.rm = T)) |>
  ggplot(aes(x = one_dose_centered, fill = Population)) +
  geom_histogram(alpha = .5) +
  geom_vline(aes(xintercept = centered_60, color = Population)) +
  labs(x = "Distance from the Group Mean (percentage points)", y = "Count")

## Centered and Scaled Distributions
scaled_60 <- covid_long |>
  group_by(Population) |>
  summarize(scaled_60 = (60 - mean(one_dose_pct, na.rm = T))/
              sd(one_dose_pct, na.rm = T)); scaled_60

covid_long |>
  group_by(Population) |>
  mutate(one_dose_z = (one_dose_pct - mean(one_dose_pct, na.rm = T))/
           sd(one_dose_pct, na.rm = T),
         scaled_60 = (60 - mean(one_dose_pct, na.rm = T))/
           sd(one_dose_pct, na.rm = T)) |>
  ggplot(aes(x = one_dose_z, fill = Population)) +
  geom_histogram(alpha = .5) +
  geom_vline(aes(xintercept = scaled_60, color = Population)) +
  labs(x = "Z-Score (standard deviations from the group mean)", y = "Count")
```

# COVID-19 Vaccine: Development Time




Sources: Pfizer and Vaccines Europe (typical); Gavi (mumps); FDA emergency use authorization, Dec 11, 2020 (COVID-19)

# Many-to-Many Joins

- If a key repeats in both x and y (a many-to-many join), every combination of matching rows is kept (a Cartesian product).
  - For example, join on country only. dplyr gives a warning:

```
> gapminder |>
    left_join(mortality_long, by = "country") |> # year is not in the key
    nrow()
[1] 77160
Warning message:
In left_join(gapminder, mortality_long, by = "country") :
  Detected an unexpected many-to-many relationship between `x` and `y`.
i Row 1 of `x` matches multiple rows in `y`.
i Row 50 of `y` matches multiple rows in `x`.
i If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.
```

- Each country has 12 rows in gapminder and 49 in mortality_long, so a country in both tables becomes $12 \times 49 = 588$ rows.
  - With `by = c("country", "year")`, we get the 1,704 rows of gapminder.
- Set `relationship = "many-to-one"` to make R check the keys.
  - The join stops with an error if a row of x matches more than one row of y.